In [1]:

```
# ============================================================
# CAPSTONE PROJECT: FMA MUSIC GENRE CLASSIFICATION
# NOTEBOOK 01: METADATA UNDERSTANDING
# PURPOSE:
# This notebook loads and explores the FMA metadata files.
# It checks the available datasets, genres, splits, subsets,
# and Echonest feature coverage.
# ============================================================
```

In [2]:

```
# ============================================================
# 1. IMPORT REQUIRED LIBRARIES
# ============================================================

import pandas as pd
import os
```

In [3]:

```
# ============================================================
# 2. SET FILE PATHS
# ============================================================
# These paths assume this notebook is saved inside:
# Capstone_FMA_Project/notebooks/
#
# The metadata files should be inside:
# Capstone_FMA_Project/data/raw/metadata/

metadata_path = "../data/raw/metadata"

tracks_path = os.path.join(metadata_path, "tracks.csv")
genres_path = os.path.join(metadata_path, "genres.csv")
features_path = os.path.join(metadata_path, "features.csv")
echonest_path = os.path.join(metadata_path, "echonest.csv")
```

In [4]:

```
# ============================================================
# 3. CHECK IF FILES EXIST
# ============================================================
# This helps confirm that the file paths are correct before loading.

print("Checking files...\n")

for file_path in [tracks_path, genres_path, features_path, echonest_path]:
    if os.path.exists(file_path):
        print(f"FOUND: {file_path}")
```

```
    else:
        print(f"NOT FOUND: {file_path}")
```

```
Checking files...

FOUND: ../data/raw/metadata\tracks.csv
FOUND: ../data/raw/metadata\genres.csv
FOUND: ../data/raw/metadata\features.csv
FOUND: ../data/raw/metadata\echonest.csv
```

In [5]:

```
# ============================================================
# 4. LOAD THE FMA METADATA FILES
# ============================================================
# tracks.csv has two header rows, so we use header=[0, 1].
# features.csv and echonest.csv have three header rows, so we use header=[0, 1, 2].
# index_col=0 means the first column is used as the track_id.

tracks = pd.read_csv(
    tracks_path,
    header=[0, 1],
    index_col=0
)

genres = pd.read_csv(genres_path)

features = pd.read_csv(
    features_path,
    header=[0, 1, 2],
    index_col=0
)

echonest = pd.read_csv(
    echonest_path,
    header=[0, 1, 2],
    index_col=0
)
```

In [6]:

```
# ============================================================
# 5. DISPLAY BASIC DATASET SHAPES
# ============================================================
# Shape means: number of rows and columns.
# Rows usually represent tracks.
# Columns represent metadata fields or features.

print("\nDataset Shapes:")
print("Tracks:", tracks.shape)
print("Genres:", genres.shape)
print("Features:", features.shape)
print("Echonest:", echonest.shape)
```

```
Dataset Shapes:
Tracks: (106574, 52)
Genres: (163, 5)
Features: (106574, 518)
Echonest: (13129, 249)
```

In [7]:

```
# ============================================================
# 6. VIEW THE FIRST FEW ROWS OF TRACKS
# ============================================================
# This lets you inspect the structure of tracks.csv.
# You should see grouped columns such as:
# album, artist, set, track.

tracks.head()
```

Out[7]:

track_id	comments	date_created	date_released	engineer	favorites	id	information	liste
2	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		60
3	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		60
5	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		60
10	0	2008-11-26 01:45:08	2008-02-06 00:00:00	NaN	4	6	NaN	476
20	0	2008-11-26 01:45:05	2009-01-06 00:00:00	NaN	2	4	"spiritual songs" from Nicky Cook	27

5 rows × 52 columns

In [8]:

```
# ============================================================
# 7. VIEW THE FIRST FEW ROWS OF GENRES
```

```
# ============================================================
# genres.csv explains the genre hierarchy.
# It contains genre IDs, parent genres, top-level genres,
# genre names, and number of tracks.

genres.head()
```

Out[8]:

	genre_id	#tracks	parent	title	top_level
0	1	8693	38	Avant-Garde	38
1	2	5271	0	International	2
2	3	1752	0	Blues	3
3	4	4126	0	Jazz	4
4	5	4106	0	Classical	5

In [9]:

```
# ============================================================
# 8. CHECK THE OFFICIAL TRAIN / VALIDATION / TEST SPLIT
# ============================================================
# This is one of the most important parts of the project.
# The FMA dataset already tells us which tracks are for:
# training, validation, and testing.
#
# This means we should NOT randomly split the data ourselves.

print("\nTrain / Validation / Test Split:")
print(tracks[("set", "split")].value_counts())
```

```
Train / Validation / Test Split:
(set, split)
training      84353
test          11263
validation    10958
Name: count, dtype: int64
```

In [10]:

```
# ============================================================
# 9. CHECK THE AVAILABLE FMA SUBSETS
# ============================================================
# The subset column tells us whether a track belongs to:
# small, medium, large.
#
# We will start with the small subset because it is easier
# to test the pipeline and models.

print("\nFMA Subsets:")
print(tracks[("set", "subset")].value_counts())
```

```
FMA Subsets:
(set, subset)
large     81574
medium    17000
small      8000
Name: count, dtype: int64
```

In [11]:

```
# ============================================================
# 10. FILTER TO THE SMALL SUBSET
# ============================================================
# We begin with FMA small because it is easier to handle.
# After the pipeline works, we can move to medium or large.

tracks_small = tracks[tracks[("set", "subset")] == "small"]

print("\nSmall Subset Shape:")
print(tracks_small.shape)
```

```
Small Subset Shape:
(8000, 52)
```

In [12]:

```
# ============================================================
# 11. CHECK GENRE LABELS IN THE SMALL SUBSET
# ============================================================
# genre_top is the main label we will predict.
# This is the target variable for classification.

small_genre_labels = tracks_small[("track", "genre_top")]

print("\nTop Genres in Small Subset:")
print(small_genre_labels.value_counts())
```

```
Top Genres in Small Subset:
(track, genre_top)
Hip-Hop          1000
Pop              1000
Folk             1000
Experimental     1000
Rock             1000
International    1000
Electronic       1000
Instrumental     1000
Name: count, dtype: int64
```

In [13]:

```
# ============================================================
# 12. CHECK IF THERE ARE MISSING GENRE LABELS
# ============================================================
# Missing labels cannot be used for supervised classification.
```

```
# We check this before modelling.

missing_labels = small_genre_labels.isna().sum()

print("\nMissing genre labels in small subset:", missing_labels)
```

```
Missing genre labels in small subset: 0
```

In [14]:

```
# ============================================================
# 13. MATCH FEATURES TO SMALL SUBSET TRACKS
# ============================================================
# features.csv contains pre-computed audio features for tracks.
# These are structured numeric features that we can use for
# Logistic Regression, Random Forest, SVM, etc.

features_small = features.loc[tracks_small.index]

print("\nFeatures for Small Subset:")
print(features_small.shape)
```

```
Features for Small Subset:
(8000, 518)
```

In [15]:

```
# ============================================================
# 14. CHECK TRAIN / VALIDATION / TEST COUNTS FOR SMALL SUBSET
# ============================================================
# This confirms the official split inside the small subset.

print("\nSplit Counts for Small Subset:")
print(tracks_small[("set", "split")].value_counts())
```

```
Split Counts for Small Subset:
(set, split)
training      6400
validation     800
test           800
Name: count, dtype: int64
```

In [16]:

```
# ============================================================
# 15. CREATE BASIC X, y, AND split VARIABLES
# ============================================================
# X = input features
# y = target labels/genres
# split = whether the row is training, validation, or test

X_small = features_small
y_small = tracks_small[("track", "genre_top")]
split_small = tracks_small[("set", "split")]
```

```
print("\nX shape:", X_small.shape)
print("y shape:", y_small.shape)
print("split shape:", split_small.shape)
```

```
X shape: (8000, 518)
y shape: (8000,)
split shape: (8000,)
```

In [17]:

```
# ============================================================
# 16. REMOVE ROWS WITH MISSING LABELS
# ============================================================
# This ensures that every row used for modelling has a genre label.

valid_rows = y_small.notna()

X_small_clean = X_small[valid_rows]
y_small_clean = y_small[valid_rows]
split_small_clean = split_small[valid_rows]

print("\nCleaned Dataset:")
print("X:", X_small_clean.shape)
print("y:", y_small_clean.shape)
print("split:", split_small_clean.shape)
```

```
Cleaned Dataset:
X: (8000, 518)
y: (8000,)
split: (8000,)
```

In [18]:

```
# ============================================================
# 17. CREATE TRAINING, VALIDATION, AND TEST SETS
# ============================================================
# We use the official FMA split.
# training = used to train the model
# validation = used to tune/check the model
# test = used for final evaluation

X_train = X_small_clean[split_small_clean == "training"]
y_train = y_small_clean[split_small_clean == "training"]

X_val = X_small_clean[split_small_clean == "validation"]
y_val = y_small_clean[split_small_clean == "validation"]

X_test = X_small_clean[split_small_clean == "test"]
y_test = y_small_clean[split_small_clean == "test"]

print("\nFinal Modelling Sets:")
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
print("X_val:", X_val.shape)
```

```
print("y_val:", y_val.shape)
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

```
Final Modelling Sets:
X_train: (6400, 518)
y_train: (6400,)
X_val: (800, 518)
y_val: (800,)
X_test: (800, 518)
y_test: (800,)
```

In [19]:

```
# ============================================================
# 18. CHECK ECHONEST COVERAGE
# ============================================================
# echonest.csv contains extra high-level audio descriptors.
# However, it does NOT cover all tracks.
#
# We check how many small-subset tracks also have Echonest data.

common_echonest_tracks = tracks_small.index.intersection(echonest.index)

print("\nEchonest Coverage:")
print("Small subset tracks:", len(tracks_small))
print("Small subset tracks with Echonest data:", len(common_echonest_tracks))
```

```
Echonest Coverage:
Small subset tracks: 8000
Small subset tracks with Echonest data: 1294
```

In [20]:

```
# ============================================================
# 19. CREATE OPTIONAL ECHONEST DATASET
# ============================================================
# This is optional and can be used later for an extra experiment.
# It will help us compare:
# 1. features.csv only
# 2. echonest.csv only
# 3. features.csv + echonest.csv

tracks_small_echonest = tracks_small.loc[common_echonest_tracks]
echonest_small = echonest.loc[common_echonest_tracks]

y_echonest = tracks_small_echonest[("track", "genre_top")]
split_echonest = tracks_small_echonest[("set", "split")]

valid_echonest_rows = y_echonest.notna()

X_echonest_clean = echonest_small[valid_echonest_rows]
y_echonest_clean = y_echonest[valid_echonest_rows]
split_echonest_clean = split_echonest[valid_echonest_rows]
```

```
print("\nOptional Echonest Dataset:")
print("X_echonest:", X_echonest_clean.shape)
print("y_echonest:", y_echonest_clean.shape)
```

```
Optional Echonest Dataset:
X_echonest: (1294, 249)
y_echonest: (1294,)
```

In [21]:

```
# ============================================================
# 20. SUMMARY OF WHAT THIS NOTEBOOK CONFIRMED
# ============================================================

print("\n==================== SUMMARY ====================")
print("1. Metadata files loaded successfully.")
print("2. tracks.csv contains the official train/validation/test split.")
print("3. tracks.csv contains subset labels: small, medium, large.")
print("4. features.csv can be used for structured modelling.")
print("5. genre_top will be used as the target variable.")
print("6. echonest.csv is optional because it only covers some tracks.")
print("7. The small subset is ready for the first structured model.")
print("================================================")
```

```
==================== SUMMARY ====================
1. Metadata files loaded successfully.
2. tracks.csv contains the official train/validation/test split.
3. tracks.csv contains subset labels: small, medium, large.
4. features.csv can be used for structured modelling.
5. genre_top will be used as the target variable.
6. echonest.csv is optional because it only covers some tracks.
7. The small subset is ready for the first structured model.
================================================
```